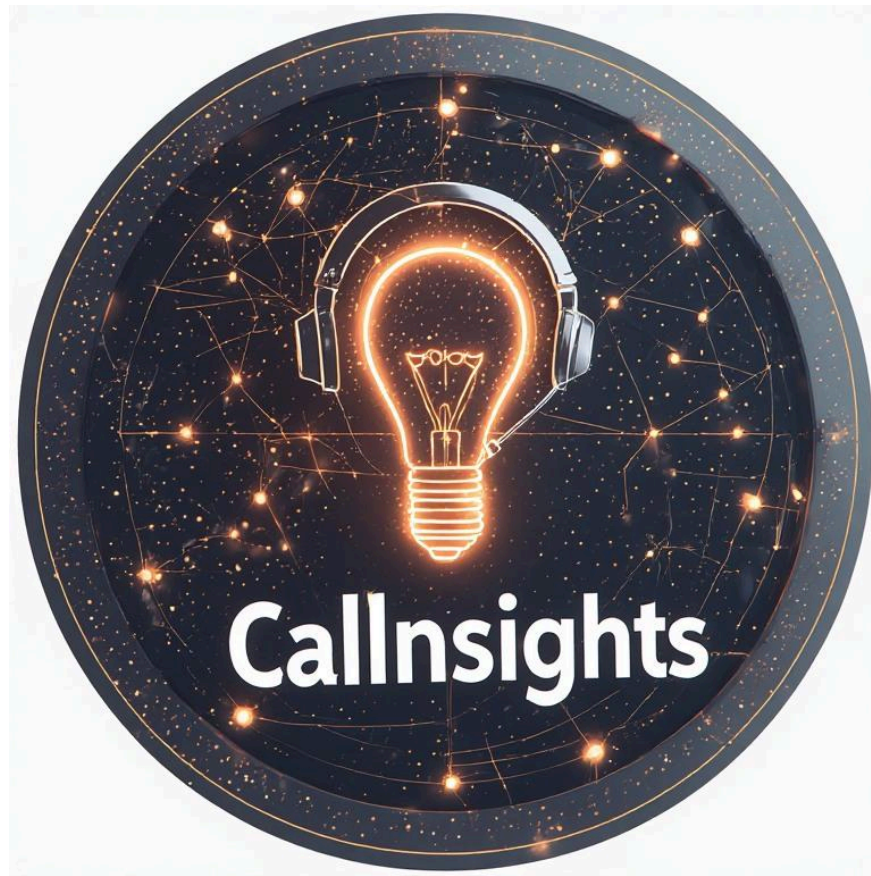




Senior Capstone: Design Document

Call Insights (CI): A Call Transcript Analysis Framework

Team Members: Abdulrahman Al Hattali, Alex Held, Anshul Kasera, and Prachi Aswani

Table of Contents

1. Executive Summary
2. Background
 - 2.1. Overview of the Idea Space
 - 2.2. Similar Ideas
 - 2.3. Required Technology
 - 2.4. Assets and Engines
 - 2.5. Software and Hardware Requirements
3. Requirements Analysis
 - 3.1. System Architecture
 - 3.2. Personnel
 - 3.3. System Features
4. Software Engineering Tools and Techniques
 - 4.1. Development Methodology (Agile vs. Traditional)
 - 4.2. Development Tools
 - 4.3. Testing Strategy and Tools
 - 4.4. Version Control and Collaboration
 - 4.5. Issue Tracking and Bug Management
 - 4.6. Code Review Process
 - 4.7. Documentation Practices
 - 4.8. Team Communication
 - 4.9. Team Meetings
 - 4.10. Continuous Integration and Deployment (CI/CD)
 - 4.11. AI Usage
5. Timeline
6. Appendix A : UI Sketches
7. Appendix B: Use Cases
8. Appendix C: Revisions
9. Appendix D: Privacy, Data use and Ethics

1. Executive Summary

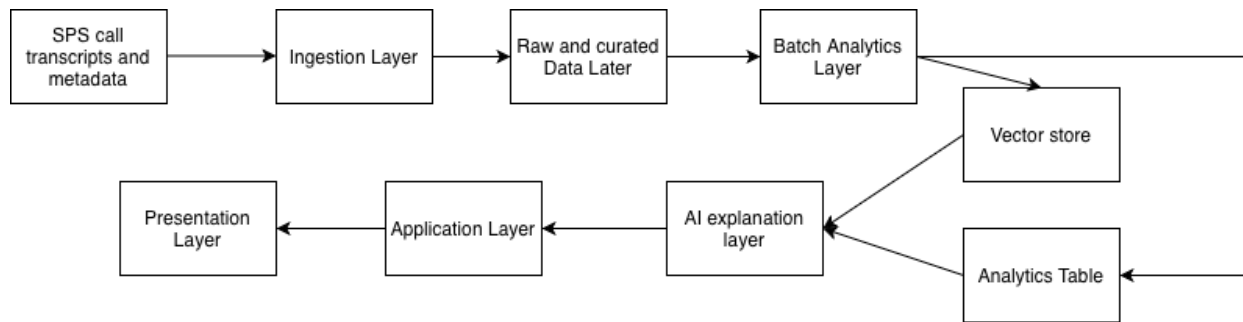
Customer service call centers generate vast amounts of conversational data, yet most organizations are only able to analyze a small fraction of it due to time, staffing, and tooling constraints. As a result, important insights about customer needs, agent effectiveness, and operational inefficiencies often go unnoticed. This project proposes Call Insights (CI), a modular analytics framework that transforms raw customer service call transcripts into actionable, evidence-based insights which will be clearly relayed over several dashboards for operational decision-makers.

CI is designed as a batch-driven call analysis system that processes historical call transcripts to identify recurring customer issues, agent behaviors, and call outcomes. Unlike traditional quality assurance approaches that rely on manual review or surface-level metrics, CI analyzes calls at scale and recognizes that a single call may involve multiple issues, multiple agent strategies, and mixed outcomes. By segmenting calls into issue-specific components, CI enables more precise analysis of what happens during customer interactions.

The system is primarily intended for call center managers, operations analysts, and strategy teams who need reliable insight into which agent behaviors lead to positive outcomes and which customer issues remain unresolved or are resolved inconsistently. CI correlates customer intent, agent actions, and outcomes across thousands of calls to surface statistically supported patterns. These patterns are presented alongside transparent traceable evidence as to how they were reached, allowing stakeholders to understand not only *what* insights are produced, but *why* they are trustworthy.

A central goal of CI is to support closed-loop learning. Insights generated by the system can inform operational strategies, such as changes to agent training or call handling guidelines, which are then tracked over time to evaluate their effectiveness. This feedback loop allows organizations to move beyond anecdotal decision-making toward continuous, data-driven improvement.

CI is being developed as a secure, self-contained framework suitable for enterprise environments, with no reliance on external APIs or live internet access. This allows the system to work with private customer data, ensuring customer data won't be leaked. By prioritizing modularity, interpretability, and evidence-first analysis, CI aims to provide a practical and extensible foundation for large-scale call intelligence rather than a single-purpose analytics tool.



CallInsights System Architecture: A high-level overview of the data pipeline and modular layers.

2. Background

Overview of the Idea Space

Customer service organizations rely heavily on phone calls to resolve customer issues, yet these interactions are difficult to analyze at scale. Manual call reviews are expensive and inconsistent, while traditional metrics such as call duration or resolution codes often fail to capture nuance in what actually happened during the conversation. As a result, organizations struggle to answer key questions such as why customers call, which agent behaviors are effective, and which issues lead to repeat calls.

The CI framework is motivated by the need to extract structured, actionable information from unstructured call transcripts. Rather than treating calls as single, monolithic events, CI models calls as sequences of issue-specific interactions, each with its own intent, behavior, and outcome. This approach enables more accurate analysis of complex calls and supports a deeper understanding of customer experience and agent performance.

The primary users of CI include agent managers, call center leadership, quality assurance teams, and operational analysts within enterprise service organizations such as SPS. These users require insights that are scalable, explainable, and directly tied to measurable outcomes. CI addresses this need by combining large-scale transcript processing with transparent analytical methods and clearly documented evidence.

Similar Ideas

Several commercial platforms offer call analytics and conversation intelligence capabilities. These systems typically provide sentiment analysis, keyword detection, or agent scoring dashboards. While effective for high-level monitoring, many existing solutions function as black-box systems, offering limited visibility into how insights are generated or how they relate to long-term outcomes.

In contrast, CI emphasizes:

- Explicit modeling of customer issues, agent behaviors, and outcomes
- Evidence-backed insights with traceable call examples
- Support for longitudinal analysis and strategy evaluation
- Deployment within data privacy focused, enterprise-controlled environments

Rather than competing as a full-scale commercial product, CI is designed to function as a framework that demonstrates how call intelligence can be implemented in a principled, extensible, and auditable way.

Feature	Traditional Call Analytics	Call Insights (CI) Framework
Analysis Model	Monolithic: Treats a call as a single event with one sentiment or outcome score.	Multi-Pattern: Segments calls into specific issues, each with its own intent, behavior, and outcome.
Transparency	Black-Box: Provides scores (e.g., "Agent Score: 8/10") without clear derivation.	Evidence-First: Every insight is backed by an "Evidence Pack" with traceable transcript examples and IDs.
Logic Engine	Surface-Level: Primarily relies on keyword detection or basic sentiment.	Triple Engine: Correlates specific Customer Problems + Agent Behaviors + Outcomes at scale.
Security	Cloud-Dependent: Often requires external APIs or internet-facing platforms.	SPS-Contained: Fully hosted on-premise; no external APIs or internet access to ensure data privacy.
Operational Impact	Passive Monitoring: Provides retrospective dashboards for high-level monitoring.	Closed-Loop Learning: Directly tracks the adoption and effectiveness of new operational strategies over time.
Architecture	Proprietary/Rigid: Difficult to customize or add new detection logic.	Modular/Pluggable: Allows SPS to add or modify analysis components (NLP/ML) without rebuilding the system.

Required Technology

To achieve its goals, CI relies on a combination of established software technologies:

- **Natural Language Processing (NLP):**
Used for transcript segmentation, topic identification, sentiment analysis, and embedding generation.
- **Machine Learning & Statistical Analysis:**
Clustering, classification, and correlation analysis are used to identify recurring patterns and measure relationships between behaviors and outcomes.
- **Data Processing Pipelines:**
Batch processing pipelines handle large volumes of transcripts and intermediate analytical artifacts.
- **Backend Services & Storage:**
Databases and file storage systems maintain transcripts, derived features, and evidence metadata.
- **Visualization & Reporting Tools:**
Dashboards and reports present insights, trends, and supporting evidence to end users.

The project primarily leverages technologies and skills already familiar to the team, minimizing the need for extensive new learning during the development phase.

Assets and Engines

CI will be built primarily from scratch as a custom analytics framework, while leveraging open-source libraries for NLP, machine learning, and data processing. Pre-existing call transcripts and metadata provided by the partner organization serve as the primary dataset for development and evaluation.

Key reusable assets include:

- Open-source NLP and ML libraries
- Existing transcript storage formats
- Internal call metadata for grounding and validation

No proprietary third-party analytics engines are required, ensuring flexibility and transparency in system design.

Software and Hardware Requirements

Developer / Software Requirements

- A modern development environment with support for Python and data science libraries
- Closed ecosystem, no internet dependent APIs / libraries to ensure data privacy
- Access to anonymized call transcripts for development and testing
- Using SPS's existing stack / workflow where applicable for easier integration

User Requirements

- Access to CI dashboards and reports through a standard web browser
- No specialized hardware beyond typical enterprise workstations for users

Hardware Requirements

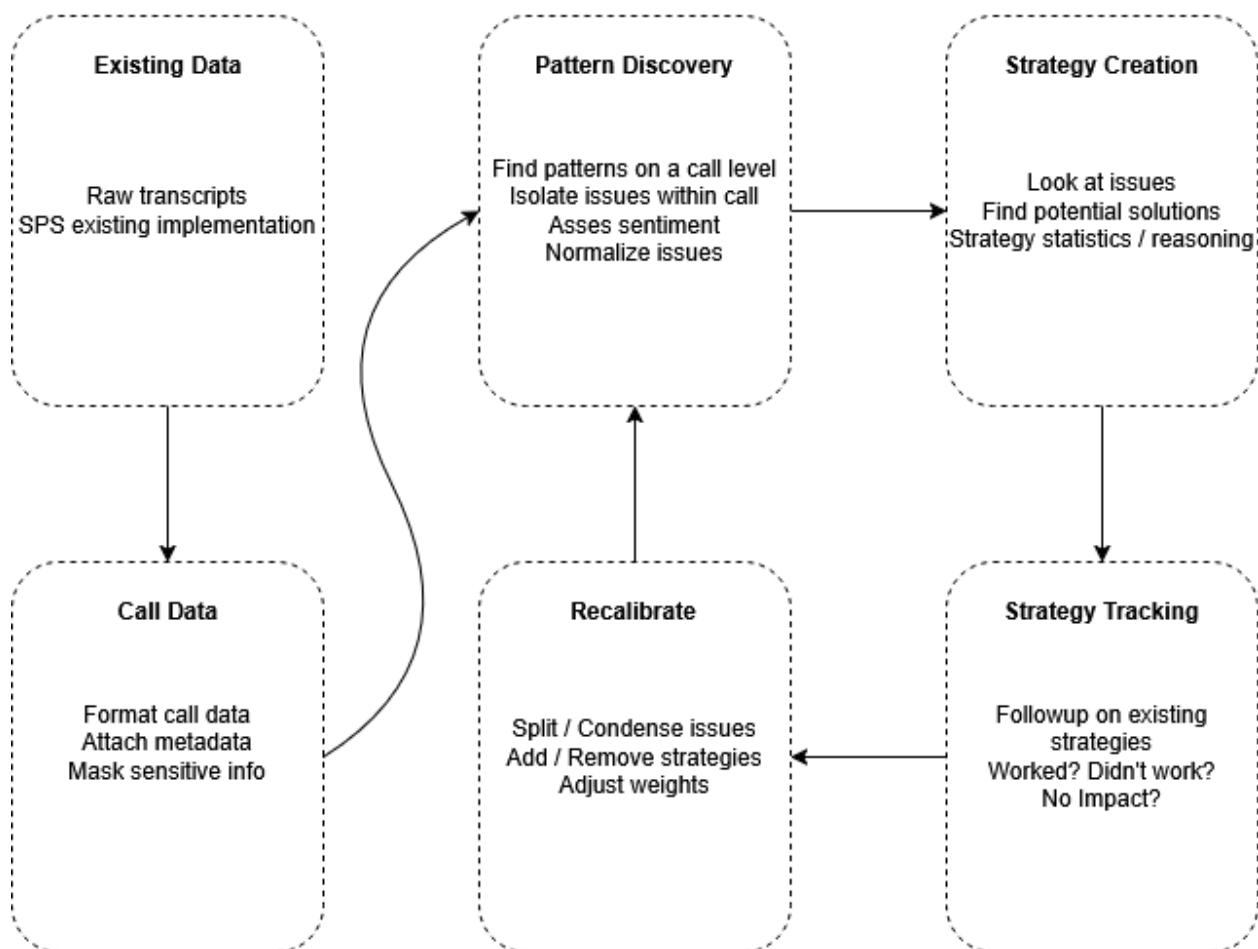
- Access to compute resources for running batch processing (provided by SPS)

3. Requirements Analysis

3.1 System Architecture

The Call Insights (CI) platform is designed as a modular, web-based intelligence system that transforms raw call transcripts into actionable, evidence-backed operational strategies. The architecture follows a 7-layer intelligence framework that transforms raw call transcripts into evidence-backed operational strategies through AI analytics, behavioral modeling, and continuous learning. The system operates as a closed-loop intelligence cycle:

High Level Overview



Each layer is modular and communicates through structured APIs and a centralized data store, enabling scalability, extensibility, and clear separation of concerns.

Layer 1: Data Ingestion Layer

This layer is responsible for collecting and preparing raw call data.

Key responsibilities:

- Ingest call transcripts from internal sources
- Attach metadata (call ID, timestamp, agent ID)
- Mask sensitive customer information
- Store masked transcripts securely
- Store related information for traceability

This layer ensures that all incoming data is standardized and privacy-compliant before processing.

Layer 2: Transcript Processing Layer (Fact Gathering Engine)

This layer transforms raw transcripts into structured representations.

Key components include:

- **Call Segmentation Engine:** Splits calls into multiple issue-based segments
- **Transcript Structuring:** Identifies core questions:
 - Why did the customer call?
 - What did the agent do?
 - What did the agent ask the customer to do?
 - What was left unaddressed?
- **Sentiment Tracking:** Captures sentiment at opening, mid-call, and closing stages

Each segment is enriched with:

- Topic classification
- Sentiment trajectory
- Resolution status
- Agent behaviors
- Unaddressed issues

Layer 3: Behavioral & Outcome Extraction Layer

This layer extracts deeper semantic signals from each call segment.

Key functionalities:

- **Issue Classification:** Using topic modeling, clustering, and supervised learning

- **Agent Behavior Detection:** Identifies actions such as empathy, escalation, or interruptions
- **Customer Action Extraction:** Determines instructions given to customers
- **Unaddressed Issue Detection:** Identifies gaps leading to repeat calls
- **Outcome Classification:** Labels segments as resolved/unresolved, escalated, and sentiment-based

All outputs are structured and stored for analysis.

Layer 4: Analytical Intelligence Layer

This is the core analytics engine that identifies patterns across large datasets.

Key components:

- **Triple Engine Analysis:**
Correlates:
 - Customer Problem
 - Agent Behavior
 - Outcome
- **Statistical Analysis:**
Computes correlation strength, lift, and significance
- **Trend Detection:**
Identifies:
 - Emerging issues
 - Volume spikes
 - Sentiment changes
 - Strategy drift
- **Knowledge Graph:**
Visualizes relationships between issues, behaviors, and outcomes
- **Alerting System:**
Detects anomalies and generates alerts

Layer 5: Evidence & Reporting Layer

This layer ensures transparency and explainability of insights.

Evidence Pack includes:

- Time window and filters
- Sample size
- Metrics and trends
- Representative call IDs
- Derived analytical signals

This enables users to validate insights with real data.

Layer 6: Strategy & Continuous Learning Layer

This layer converts insights into actionable improvements.

Key components:

- **Strategy Creation:** Define operational strategies
- **Lifecycle Management:** Track strategies through stages:
 - Proposed -> Accepted -> Implemented -> Evaluating -> Closed
- **Effectiveness Tracking:** Measure impact using before/after comparisons

This layer enables continuous improvement through data-driven decisions.

Layer 7: Presentation & Governance Layer (Web Application)

CI is delivered as a unified web application, not a collection of dashboards.

Key user-facing modules:

- Executive Overview
- Daily Insights Pulse
- Monthly Recalibration
- Issue Explorer
- Call Drilldown
- Strategy Management
- Strategy Effectiveness
- Reports & Export
- Sentiment Journey Map
- Voice-to-Insight
- Natural Language Query (Ask CI)
- Admin & AI Governance

This layer provides an interactive workspace where users explore insights, validate findings, and take action.

Layer 8: Pre-Call Pattern Layer (Dynamic IVR Intelligence)

This layer is implemented after sufficient learning has been achieved.

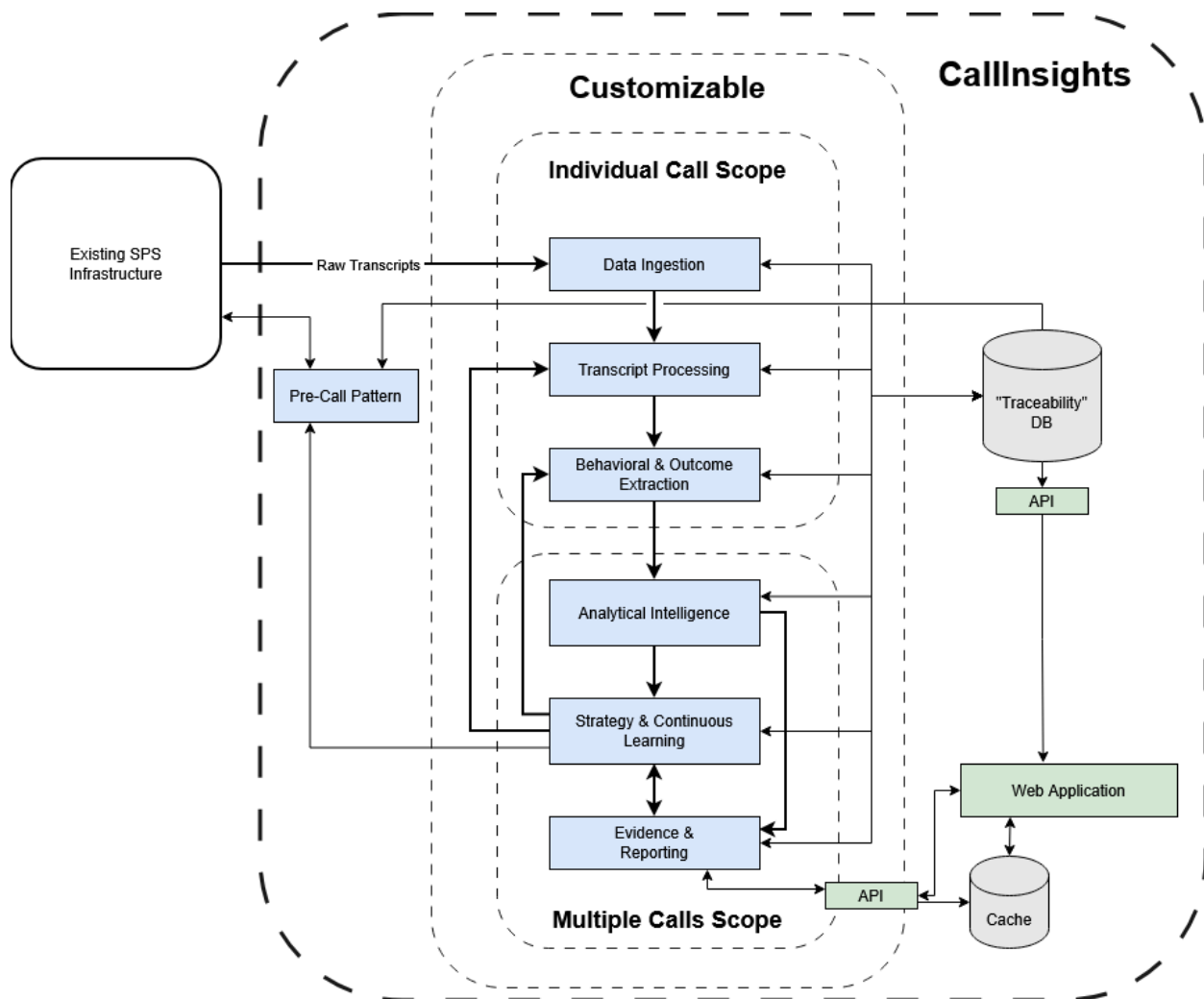
Inputs:

- Recurring issues
- Call intent distributions
- Successful resolution patterns

Outputs:

- Smarter IVR routing
- Issue-aware call tagging
- Early metadata enrichment

This layer would extend CI to be capable of upstream call handling.



Cache will primarily handle automated reports

3.2 Personnel

The development of the Call Insights (CI) platform is distributed across all team members to ensure balanced contribution across the full 7-layer intelligence framework. Rather than assigning ownership of entire layers, responsibilities are divided by subsystem and technical specialization, with each member contributing to multiple layers of the architecture.

Abdulrahman Al Hattali: Backend Infrastructure & Data Pipeline Lead

Abdulrahman is responsible for designing and implementing the core backend infrastructure that supports data flow across the CI platform.

Primary Responsibilities:

- Data Ingestion Layer (Layer 1)
 - Transcript ingestion pipeline
 - Metadata attachment and storage
 - Data masking and privacy compliance
- Backend API development for inter-layer communication
- Database schema design for transcripts, segments, and derived signals
- Integration between processing layers and frontend services

Cross-Layer Contributions:

- Supports Transcript Processing Layer (Layer 2) by optimizing data pipelines
- Enables Evidence & Reporting Layer (Layer 5) through efficient data retrieval

Justification:

Abdulrahman has strong experience in backend development and data systems, making him well-suited for handling data pipelines, APIs, and scalable infrastructure.

Alex Held - Frontend & Web Application Lead

Alex is responsible for building the unified CI web application and ensuring a seamless, interactive user experience across all presentation modules.

Primary Responsibilities:

- Presentation & Governance Layer (Layer 7)
 - Core web application architecture
 - Navigation and interaction design
 - Implementation of all UI modules:
 - Executive Overview
 - Issue Explorer
 - Call Drilldown
 - Strategy Management
- Integration of frontend with backend APIs

Cross-Layer Contributions:

- Works with Analytical Layer (Layer 4) to visualize insights
- Supports Strategy Layer (Layer 6) through interactive UI tools
- Contributes to Voice-to-Insight and Ask CI interfaces

Justification:

Alex has experience in frontend development and UI/UX design, making him well-equipped to create an intuitive and visually engaging web application.

Anshul Kasera - Analytics & Intelligence Engine Lead

Anshul is responsible for implementing the core analytical, statistical, and AI-driven components that power CI's intelligence capabilities, including contributions to NLP system development and integration.

Primary Responsibilities:**Analytical Intelligence Layer (Layer 4)**

- Triple Engine (Problem, Behavior, Outcome correlation)
- Statistical computations (correlation, lift, significance)
- Trend detection (spikes, anomalies, drift)
- Knowledge Graph construction and analysis
- Alerting system for anomaly detection

AI & NLP Implementation Contributions (Layer 2 and Layer 3)

- Co-implementation of NLP pipelines for transcript processing and feature extraction
- Integration of sentiment analysis, issue classification, and outcome detection outputs into the analytics engine
- Development of models that leverage both structured data and NLP-derived features
- Feature engineering from text-based and structured data sources
- Model evaluation, performance tuning, and validation across AI components

Cross-Layer Contributions:

- Supports Behavioral & Outcome Extraction Layer (Layer 3) through both model development and NLP integration
- Enables Strategy Layer (Layer 6) through insight generation
- Provides data for Evidence Packs (Layer 5)
- Ensures seamless flow of NLP-derived signals into downstream intelligence systems

Justification:

Anshul's background in machine learning, statistical analysis, NLP and data modeling enables him to contribute not only to CI's analytical backbone but also to the implementation and integration of NLP-driven intelligence within the platform.

Prachi Aswani - NLP, AI Systems & Product Integration Lead

Prachi is responsible for the natural language processing components and advanced AI-driven features that enable CI's deep understanding of call transcripts and user interaction.

Primary Responsibilities:

- Transcript Processing Layer (Layer 2)
 - Call segmentation
 - Sentiment analysis over time
- Behavioral & Outcome Extraction Layer (Layer 3)
 - Issue classification
 - Agent behavior detection
 - Outcome classification
- AI-driven features:
 - Natural Language Query (Ask CI)
 - Voice-to-Insight pipeline

Cross-Layer Contributions:

- Supports Analytical Layer (Layer 4) by providing structured NLP outputs
- Enhances Presentation Layer (Layer 7) with intelligent interaction features
- Contributes to Pre-Call Pattern Layer (Layer 8) through learned patterns

Justification:

Prachi has strong experience in machine learning, NLP, and data analysis, along with prior work on large-scale text classification systems, making her highly qualified to lead CI's AI and language processing components.

Team Collaboration Model

All team members collaborate across layers to ensure integration and system coherence. Key collaborative areas include:

- End-to-end pipeline integration (Layer 1 to Layer 7)
- Evidence Pack generation and validation
- Strategy tracking and evaluation workflows
- Advanced features (Rank 3), including AR/VR visualization and simulation tools

This distributed ownership model ensures that:

- Each team member contributes to multiple parts of the system
- No single point of failure exists
- The system remains modular, scalable, and well-integrated

3.3 System Features

The CI platform features are categorized into three priority levels.

High-Level Feature Grid

Feature Area	Priority	Feature	Description
Data	1	Transcript Ingestion	Upload and store transcripts with metadata and masking
Processing	1	Call Segmentation	Split calls into multi-issue segments
NLP	1	Issue Classification	Identify why customers called
NLP	1	Behavior Extraction	Detecting agent actions (empathy, escalation, etc.)
NLP	1	Sentiment Analysis	Track sentiment over time
NLP	1	Outcome Classification	Label resolution and escalation status
Analytics	1	Triple Engine	Correlate problem, behavior, and outcome
UI	1	Core Web Interface	Navigation and basic data viewing

Feature Area	Priority	Feature	Description
Insights	2	Issue Explorer	Interactive analysis of issue patterns
Insights	2	Call Drilldown	View transcripts with extracted signals
Insights	2	Daily Insights Feed	Show emerging issues and anomalies
Strategy	2	Strategy Management	Create and track strategies
Strategy	2	Effectiveness Tracking	Measure impact of strategies
Reporting	2	Evidence Pack Export	Generate structured reports
AI	2	Natural Language Query (Ask CI)	Query system using plain English
Voice	2	Voice-to-Insight	Convert speech to analyzed transcript
Alerts	2	Smart Alerts	Notify users of anomalies

Feature Area	Priority	Feature	Description
Visualization	3	Knowledge Graph	Visualize relationships between variables
AI	3	Predictive Analytics	Predict escalation and repeat calls
AI	3	Strategy Recommendations	Suggest optimal strategies
Simulation	3	Call Simulation Tool	Simulate real-time call analysis
XR	3	AR/VR Insight Visualization	Explore insights in immersive environments
Collaboration	3	Insight Annotation	Team-based commenting and tagging
Explainability	3	AI Transparency Panel	Show reasoning behind insights

4. Software Engineering Tools and Techniques

The Call Insights (CI) team will adopt a structured yet flexible software engineering approach that balances rapid prototyping with long-term system reliability. Given the complexity of the system and evolving understanding of NLP pipelines, we will follow a hybrid Agile methodology, supported by modern development, testing, and collaboration tools.

4.1 Development Methodology (Agile vs. Traditional)

The team will follow a lightweight Agile approach (Scrum-inspired) rather than a rigid traditional model. Due to the exploratory nature of NLP, machine learning, and analytics pipelines, it is not feasible to fully specify all requirements upfront.

Instead, we will:

- Plan in 2-week sprints
- Maintain a rolling backlog of features and technical tasks
- Define short-term deliverables (1-2 sprints ahead) while keeping long-term goals flexible
- Continuously refine requirements based on prototype results and stakeholder feedback

This approach allows us to:

- Iterate on uncertain components (e.g., segmentation, behavior detection)
- Quickly validate assumptions
- Adapt to data and model limitations

4.2 Development Tools

IDEs and Development Environment

- IDEs: VS Code, Rider
- Language: Python (backend), JavaScript/TypeScript (frontend)
- Containerization: venv for python backend, package-lock for frontend

Core Libraries and Frameworks

- NLP: spaCy, Hugging Face Transformers
- Machine Learning: scikit-learn, PyTorch
- Backend API: FastAPI
- Frontend: React
- HTTP client requests: Axios

- Visualization: React Flow, D3.js
- Database: PostgreSQL, optional Neo4j (for knowledge graph)

API Development and Testing

- API Documentation: Swagger (via FastAPI auto-docs)
- API Testing: Postman

These tools will ensure that all backend services are well-documented, testable, and accessible to all team members.

4.3 Testing Strategy and Tools

Testing will be integrated throughout development to ensure correctness and reliability.

Testing Levels

- **Unit Testing:**
 - Tool: pytest
 - Focus: individual components (segmentation, classification, etc.)
- **Integration Testing:**
 - Validate interactions between layers (e.g., NLP -> analytics -> API)
- **System Testing**
 - Validate full workflows (e.g., transcript -> insight -> strategy)
- **Manual Testing:**
 - UI workflows and user interaction validation

Data Validation

- Special attention will be given to:
 - Model outputs
 - Statistical computations
 - Edge cases in transcripts

4.4 Version Control and Collaboration

Versioning System

- Platform: GitLab
- Repository Structure:
 - ci-backend
 - ci-frontend
 - ci-ml-pipeline

Branching Strategy

- main: stable production-ready code
- dev: integration branch
- Feature branches -> feature/<name>

Merge Policy

- All changes must go through Merge Requests (MRs)
- No direct commits to main

4.5 Issue Tracking and Bug Management

- Tool: GitLab Issues
- Each task will include:
 - Description
 - Assignee
 - Priority
 - Time estimate
 - Labels

Bug Tracking

- Bugs will be labeled and prioritized
- Critical bugs addressed immediately
- Non-critical bugs scheduled into future sprints

4.6 Code Review Process

All code contributions will undergo peer review before merging.

- At least one reviewer required for approval
- Complex components (e.g., NLP models) may require team-wide review
- Focus areas:
 - Code correctness
 - Readability
 - Performance
 - Consistency with architecture

This ensures maintainability and shared understanding across the team.

4.7 Documentation Practices

Global Documentation

- Maintained in GitLab Wiki
- Includes:
 - Architecture overview
 - API documentation
 - Setup instructions
 - Design decisions

Local Documentation

- Code-level comments and docstrings
- Focus on:
 - Function purpose
 - Inputs/outputs
 - Assumptions

Documentation Approach

- We follow a “comment-as-you-build” approach, not post-hoc documentation
- Ensures documentation remains accurate and useful

4.8 Team Communication

- Primary communication: Discord
- Channels organized by:
 - Backend
 - Frontend
 - ML/NLP
 - General

Communication Principles

- Transparent updates
- Quick issue escalation
- Async-friendly collaboration

4.9 Team Meetings

- Weekly full-team meeting
 - Progress updates

- Blockers
- Sprint planning
- **Mid-week check-ins**
 - Focused technical discussions
- **Ad-hoc meetings**
 - For debugging, design decisions, or integration work
- **End-of-sprint review**
 - Demo progress
 - Reflect on improvements

4.10 Continuous Integration and Deployment (CI/CD)

- Platform: GitLab CI/CD
- Automated pipelines for:
 - Running tests
 - Linting code
 - Verifying builds

Future Extensions:

- Automated deployment (if infrastructure allows)
- Model evaluation pipelines

4.11 AI Usage

AI in Design

- Review and critique system architecture and design decisions
- Identify potential edge cases, bottlenecks, and scalability concerns
- Suggest alternative approaches for NLP pipelines, data modeling, and system structure
- Propose additional features or improvements based on current system goals

AI in Development

- Generating boilerplate or utility functions
- Assisting with debugging and error resolution
- Providing inline code suggestions during development
- Implementing well-scoped features or components
- Generating test cases for unit and integration testing

All code that is substantially generated or influenced by AI will be explicitly documented with comments indicating AI involvement.

Code Understanding Requirement

- Being able to explain the purpose and functionality of each major code segment
- Demonstrating a working understanding of underlying algorithms, libraries, and frameworks used
- Explaining how the code integrates with the broader system

AI-Assisted Testing and Validation

AI will also support testing by:

- Generating diverse and edge-case-heavy test inputs
- Assisting in writing unit and integration tests
- Suggesting validation strategies for model outputs and statistical computations

This helps improve coverage and robustness of the system, especially for complex NLP-driven components.

Responsible AI Usage

The team will ensure responsible and effective use of AI by:

- Verifying correctness of all AI-generated outputs before integration
- Avoiding blind reliance on AI-generated code or design suggestions
- Maintaining consistency with project architecture and coding standards
- Ensuring all final decisions are human-reviewed and justified

Summary

This combination of Agile methodology, modern tooling, structured collaboration, and strong testing practices ensures that the CI platform will be developed in a scalable, maintainable, and professional manner. The team's approach emphasizes iterative learning, code quality, and clear communication, all of which are critical for building a complex AI-driven system.

5. Timeline

The development plan for the next semester is divided into three phases: Alpha (Weeks 1-4), Beta (Weeks 5-8), and Final (Weeks 9-12). Each phase builds on the previous one, progressing from core functionality to full system integration and production readiness.

Alpha Stage (Weeks 1-4)

Week	Prachi Aswani	Anshul Kasera	Abdulrahman Al Hattali	Alex Held
Week 1	Implement question extraction from transcripts	Define similarity metrics for call matching	Improve transcript ingestion pipeline	Set up frontend architecture
Week 2	Improve segmentation for multi-question detection	Prototype embedding-based retrieval	Optimize database schema	Develop Executive Dashboard UI
Week 3	Integrate NLP outputs into pipeline	Build initial strategy comparison logic	Develop backend APIs	Connect frontend to backend
Week 4	Validate extraction accuracy	Run statistical validation	Pipeline testing and debugging	Implement Call Drilldown UI

Beta Stage (Weeks 5-8)

Week	Prachi Aswani	Anshul Kasera	Abdulrahman Al Hattali	Alex Held
Week 5	Refine NLP models for question detection	Improve similarity ranking model	Optimize API performance	Develop Issue Explorer UI
Week 6	Add contextual understanding of queries	Evaluate outcome correlations	Implement caching mechanisms	Build Strategy Dashboard
Week 7	Integrate NLP with analytics layer	Enhance Triple Engine analysis	Improve system scalability	Begin AR visualization prototype
Week 8	End-to-end pipeline testing	Statistical tuning and validation	System integration testing	Enhance AR interaction features

Final Stage (Weeks 9-12)

Week	Prachi Aswani	Anshul Kasera	Abdulrahman Al Hattali	Alex Held
Week 9	Optimize NLP performance	Validate insights on full dataset	Improve system stability	Finalize AR visualization
Week 10	Handle edge cases and errors	Conduct large-scale evaluation	Implement security checks	UI/UX improvements
Week 11	Documentation and integration	Final analytics validation	Deployment preparation	Presentation features
Week 12	Final system validation	Final reporting metrics	Deployment support	Demo preparation

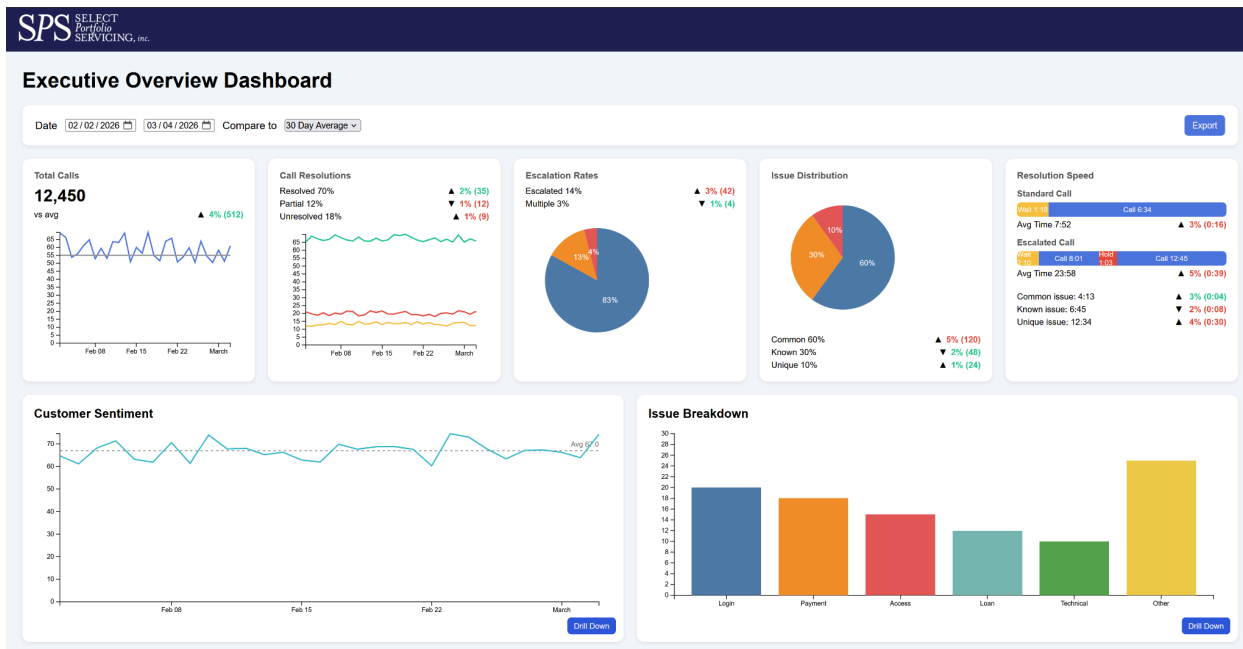
APPENDIX A: UI SKETCHES

Below is the structured list of views We have to design

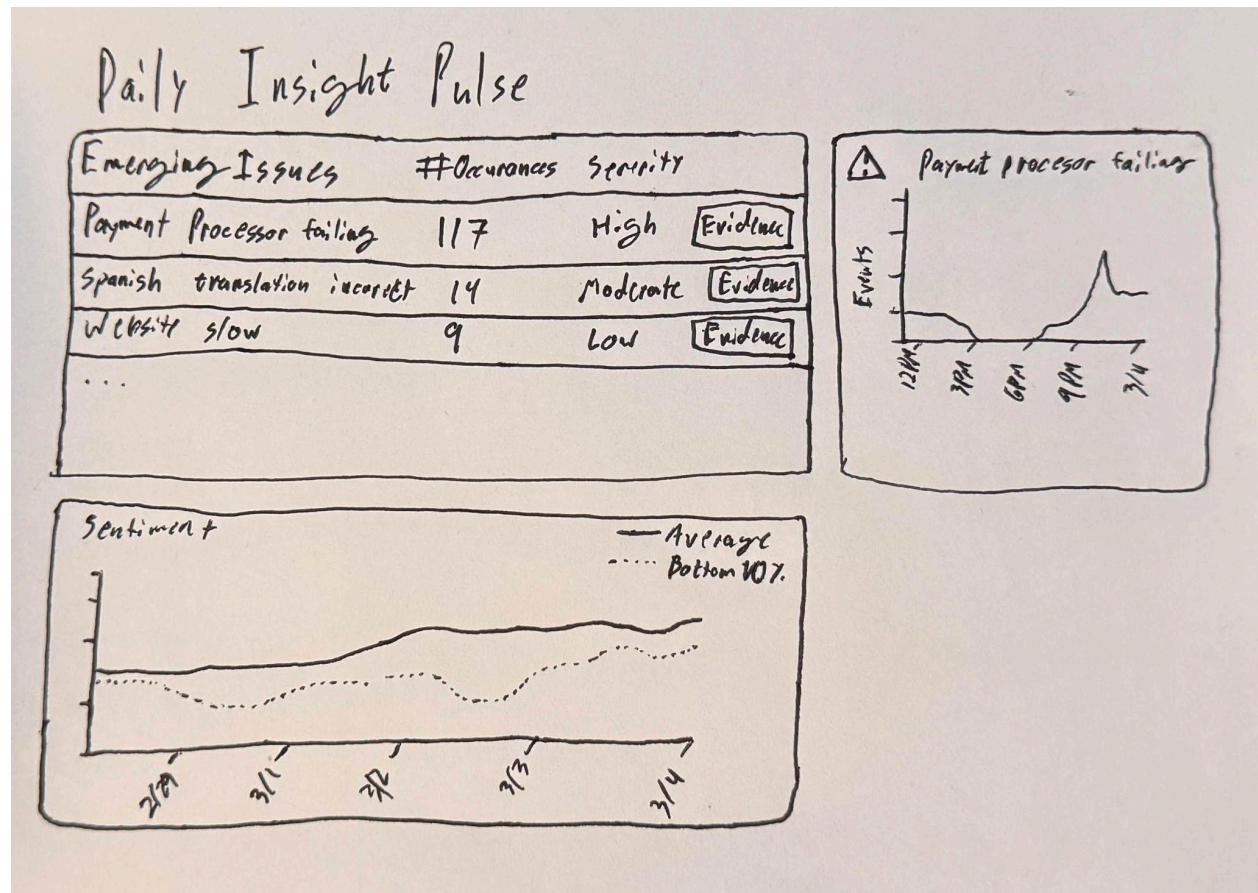
Create mockups (Figma / PowerPoint / Adobe XD) matching these.

UI-1: Executive Overview Dashboard

High-level performance snapshot.

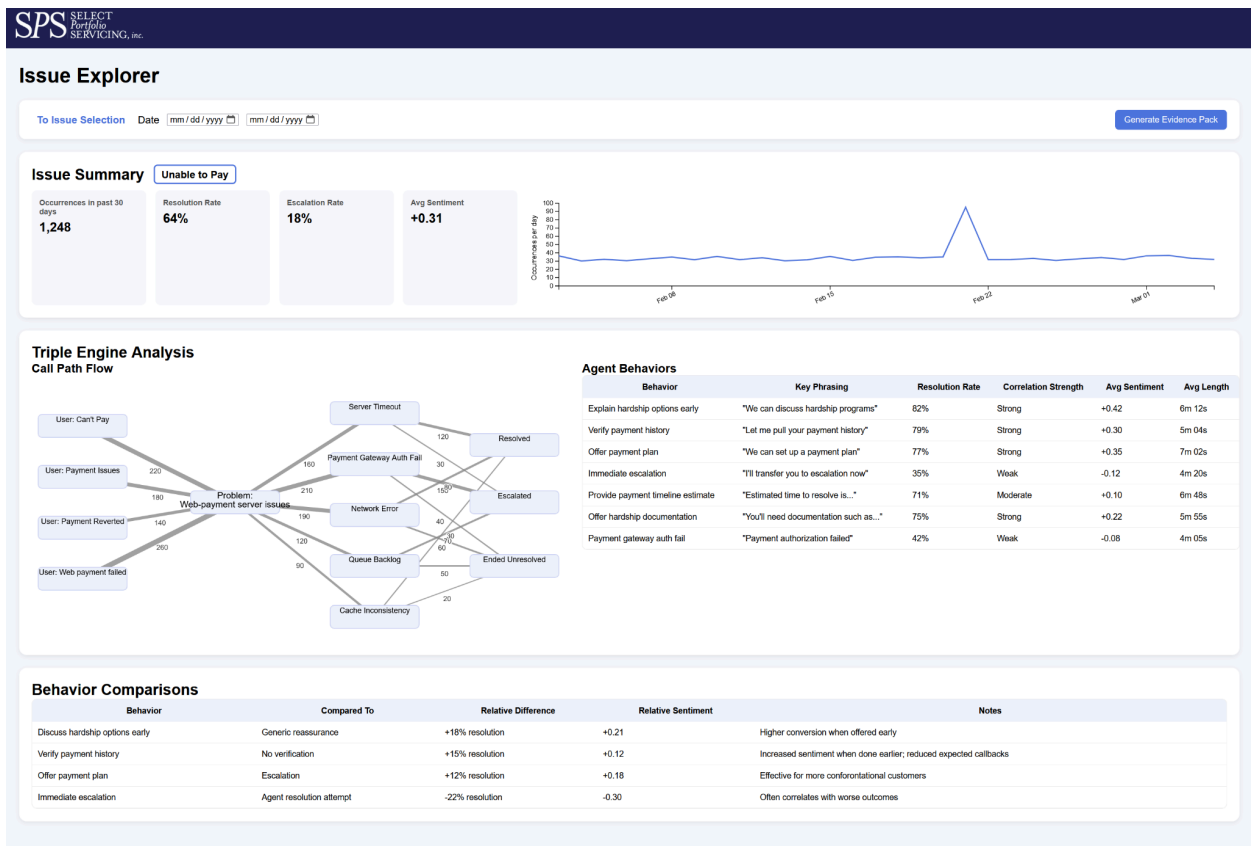


UI-2: Daily Insights Pulse



UI-3: Issue Explorer (Core Analytical Screen)

MOST IMPORTANT UI



[illegible]

Strategy Effectiveness

Strategy

Escrow Clarity Script

Date

03/06/2026

03/06/2026

Queue

Inbound - General

Export Summary

Export Evidence Pack

Before vs After (KPI Snapshot)

AHT (Avg Handle Time)

Before: 9m 10s

After: 8m 20s

-9%

FCR

Before: 62%

After: 69%

+7%

Escalation Rate

Before: 18%

After: 12%

-6%

Sentiment

Before: -0.18

After: -0.05

Improves

Tracking window: Last 30 days

Sample size: 200 calls

Trends Over Time

Adoption Rate

W1		34%
W2		47%
W3		61%
W4		74%

Outcome Metric (Escalation Rate)

W1		18%
W2		16%
W3		14%
W4		12%

Drift: Low

Insights & Evidence

What changed

- Escalations dropped after agents used the timeline explanation early.
- Sentiment improved most in Inbound - General queue.
- FCR improvement concentrated in Escrow calls.

Representative Calls

- C-11390: Explained timeline early, resolved in one call.
- C-11374: Escalation avoided after payment options review.
- C-11361: Positive sentiment shift after script adherence.
- C-11348: Customer confusion reduced via escrow breakdown.
- C-11332: Follow-up prevented repeat call within 7 days.

View Supporting Calls

Open Evidence Pack

Select a strategy + time window

Compare KPI changes before vs after

Export evidence-backed report (PDF/CSV)

UI-6: Strategy Management

SPSSELECT
Portfolio
SERVICING, inc.

Strategy Management

Date Range

03/27/2026

03/05/2026

Status

Active

Target Issue

Escrow

Owner/Team

Training

+ New Strategy

Export

Filters

Status

Active

Paused

Archived

Target Issue

Escrow

Owner/Team

Training

Date Window

Last 7 days

Last 30 days

Last 90 days

Reset Filters

Strategies

Strategy Name	Target Issue	Status	Adoption Rate	Outcome Change
Escrow Clarity Script	Escrow	Active	74%	+11%
Payment Plan Prompting	Payment	Active	68%	+8%
Complaint De-escalation Path	Complaint	Paused	41%	+2%
Hardship Options First	Payment	Active	82%	+13%
Escalation Deflection Pack	Complaint	Archived	57%	-1%
Escrow Timeline Coaching	Escrow	Active	63%	+6%
Structured Wrap-up Checklist	Complaint	Paused	49%	+3%

Filter strategies by status and issue

Track adoption and outcome changes

Detailed view of selected strategy

Selected Strategy Details

Description

Encourage agents to explain escrow changes in plain language, then confirm customer understanding before next-step recommendations.

Tracking Window

Last 30 days

Linked Evidence Pack

Evidence Pack 01

View Effectiveness

Edit Strategy

Strategies

Strategy Name	Target Issue	Status	Adoption Rate	Outcome Change
Escrow Clarity Script	Escrow	Active	74%	+11%
Payment Plan Prompting	Payment	Active	68%	+8%
Complaint De-escalation Path	Complaint	Paused	41%	+2%
Hardship Options First	Payment	Active	82%	+13%
Escalation Deflection Pack	Complaint	Archived	57%	-1%
Escrow Timeline Coaching	Escrow	Active	63%	+6%
Structured Wrap-up Checklist	Complaint	Paused	49%	+3%

Strategies

Target Issue	Status	Adoption Rate	Outcome Change	Last Updated	Actions
Escrow	Active	74%	+11%	2026-03-01	View Edit Archive
Payment	Active	68%	+8%	2026-02-27	View Edit Archive
Complaint	Paused	41%	+2%	2026-02-24	View Edit Archive
Payment	Active	82%	+13%	2026-03-02	View Edit Archive
Complaint	Archived	57%	-1%	2026-02-11	View Edit Archive
Escrow	Active	63%	+6%	2026-02-26	View Edit Archive
Complaint	Paused	49%	+3%	2026-02-20	View Edit Archive

CallInsights Design Document

31

UI-7: Reports & Export

SPS
SELECT
Portfolio
SERVICING, inc.

Reports & Export

Date

03/06/2026

03/06/2026

Issue Category

Escrow

Team

Training

Queue

Inbound - General

Sample Size (Top N)

20

Generate Evidence Pack

Download PDF

Export CSV

Evidence Pack Builder

Evidence Pack Contents

- ☒ Time window definition
- ☒ Filters applied (queue/issue/team)
- ☒ Counts & trends
- ☒ Sample size
- ☒ Representative call IDs
- ☒ Derived signals (sentiment shift)

UC-6 / CTO Export Notes

Builder includes export-ready sections for evidence pack generation and static PDF/CSV outputs.

Preview

Selected Filters

Date Range: 2026-02-01 to 2026-03-01
Filters applied: Queue = Inbound - General, Issue = Escrow, Team = QA
Sample Size: Top 20 calls

Key Metrics Snapshot

AHT 6m 42s	FCR 71%	Escalation Rate 17%	Avg Sentiment +0.26
----------------------	-------------------	-------------------------------	-------------------------------

Representative Calls

Call ID	Issue	Team	Sentiment	Resolution	Notes
C-10981	Escrow	QA	+0.39	Resolved	Clear script adherence
C-10974	Payment	Ops	+0.18	Resolved	Plan accepted
C-10966	Complaint	Training	-0.05	Escalated	Supervisor callback
C-10952	Escrow	QA	+0.31	Resolved	Timeline clarified
C-10941	Payment	Ops	+0.24	Resolved	Late fee waived
C-10933	Escrow	QA	+0.22	Resolved	Transfer to specialist

UI-8: Compliance Dashboard

SPS
SELECT
Portfolio
SERVICING, inc.

Compliance Monitoring Dashboard

From

02/05/2026

To

03/07/2026

Regulation

All

Search

Search transcripts...

Export Audit

Compliance Rate

60%

Percentage of reviewed calls that met required disclosures

Calls Reviewed

120

Total calls analyzed in selected range

Calls Missing Disclosures

48

Calls flagged for missing mandatory phrases

Top Missing Phrase

Disclose fee (17)

Most commonly omitted disclosure

Compliance Trend

● Compliant ● Non-compliant

Missing Phrase Distribution

Click a slice to filter calls

Non-compliant Calls

Showing calls missing required disclosures. Click "View" to inspect transcript.

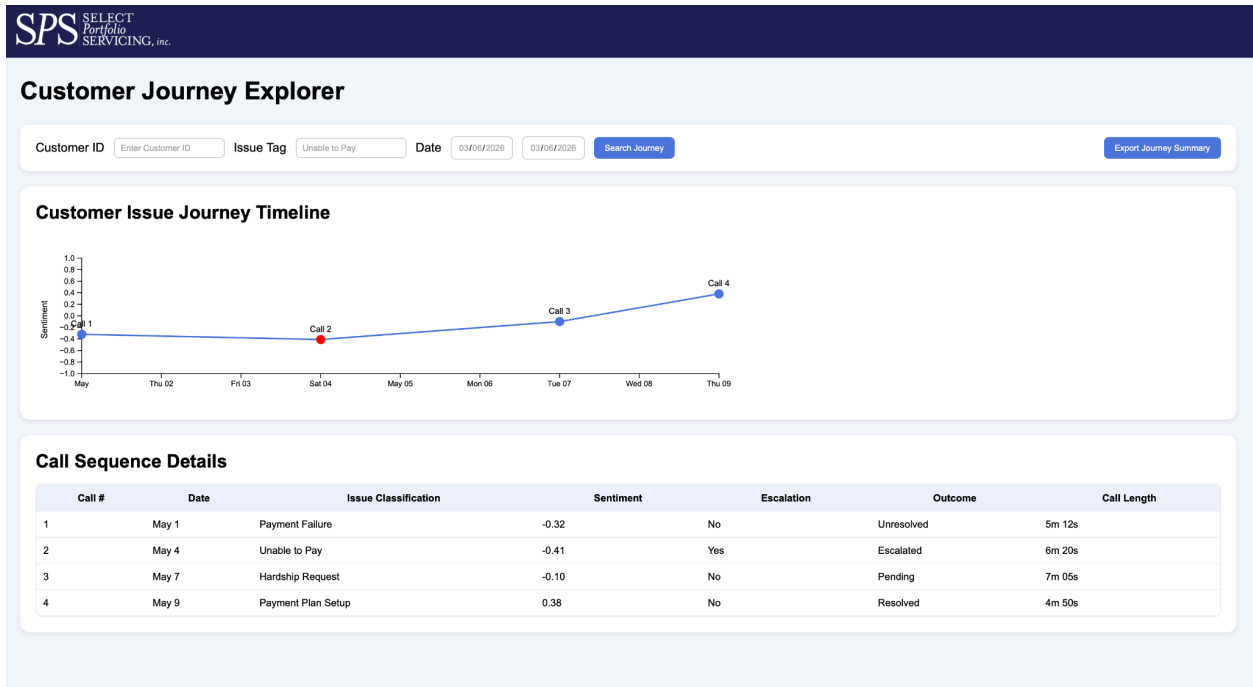
Call ID	Agent	Time	Missing Phrases	Actions
C1000	Bob	2026-02-13 00:49	Offer terms	View
C1001	Dana	2026-02-28 00:49	State rate	View
C1002	Bob	2026-03-01 00:49	State rate	View

Download CSV

CallInsights Design Document

32

UI-9: Customer Journey Explorer



APPENDIX B: UI USE CASES

Team Name: CallInsights

Team Members: Abdulrahman Al Hattali, Alex Held, Anshul Kasera, and Prachi Aswani

Use Case 1

Number: UC-1

Title: View Executive Overview Dashboard

Preparer: Prachi Aswani

Actor/User: CTO

User Story

As a CTO, I want to quickly understand the overall health of call center performance so that I can identify risks, trends, and areas needing strategic attention.

Course of Events

1. User logs into CI.
2. System displays Executive Overview Dashboard.
3. User sees:
 - Total call volume (selected time window)
 - % Resolved vs Unresolved
 - Escalation rates
 - Call issue distributions
 - Recurring issues
 - Any other immediate major problems/issues
4. User selects a time filter (e.g., Last 30 Days).
5. Dashboard updates dynamically.

6. The user clicks on a top issue to drill deeper (redirects to UC-4).

Exceptions / Alternatives

- If no data exists for the selected time window, show "Insufficient Data" state.
- If the user lacks permissions, limited metrics view.

Related UI

See UI-1: Executive Overview Dashboard

Use Case 2

Number: UC-2

Title: Analyze Daily Insights Pulse

Preparer: Prachi Aswani

Actor/User: Operations Analyst

User Story

As an operations analyst, I want to see newly emerging issues or spikes in call patterns so that I can respond quickly to operational disruptions.

Course of Events

1. The user navigates to "Daily Insights Pulse".
2. System displays:
 - Emerging issue clusters
 - Volume spikes
 - Sentiment shifts
 - Escalation anomalies

3. The user clicks an emerging issue.
4. System shows:
 - Trend graph
 - Sample call IDs
 - Sentiment trajectory distribution
5. User exports insight summary.

Exceptions

- No statistically significant changes, show “No significant anomalies detected.”

Related UI

See UI-2: Daily Insights Pulse View

Use Case 3

Number: UC-3

Title: Explore Issue Patterns (Triple Engine Analysis)

Preparer: Alex Held

Actor/User: Strategy Team Member / General Agent Manager

User Story

As a strategy team member, I want to analyze which agent behaviors correlate with positive or negative outcomes for a specific issue so that I can refine agent training strategies.

Course of Events

1. The user selects an issue (e.g., “Payment Confusion”).
2. System runs Triple Engine view:
 - Problem

- Agent Behavior
 - Outcome
3. System displays:
- Correlation strength
 - Lift
 - Statistical significance
4. User filters by:
- Agent group
 - Time window
 - Escalation status
5. System updates performance comparison.
6. User clicks “Generate Evidence Pack.”

Exceptions

- Low sample size, system displays statistical warning.

Related UI

See UI-3: Issue Explorer

See UI-5: Strategy Effectiveness Dashboard

Use Case 4

Number: UC-4

Title: Drill Down into Specific Call

Preparer: Alex Held

Actor/User: QA Manager

User Story

As a QA manager, I want to inspect a specific call transcript and its extracted metadata so that I can verify the validity of insights.

Course of Events

1. The user clicks on the call ID from the dashboard.
2. System opens Call Drilldown view.
3. User sees:
 - Full transcript (masked sensitive info)
 - Segmented chunks
 - Issue tags
 - Sentiment over time graph
 - Agent behavior markers
 - Resolution classification
4. User downloads transcript summary.

Exceptions

- If the transcript is missing, show “Transcript unavailable.”

Related UI

See UI-4: Call Drilldown View

Use Case 5

Number: UC-5

Title: Evaluate Strategy Effectiveness Over Time

Preparer: Abdulrahman Al Hattali

Actor/User: Director of Training

User Story

As a training director, I want to measure whether an accepted strategy improved outcomes over time so that I can justify continued adoption, revision, or retirement.

Course of Events

1. User navigates to Strategy Management.
2. User selects an accepted strategy (for example, “Explain Timeline Early”).
3. User clicks View Effectiveness.
4. System opens the Strategy Effectiveness Dashboard.
5. System displays:
 - Before/After comparison
 - Resolution rate change
 - Average Handle Time (AHT) change
 - First Call Resolution (FCR) change
 - Escalation rate change
 - Sentiment improvement
 - Trend over time / drift indicator
6. User reviews supporting insights and representative calls.
7. The user optionally exports a summary or opens the linked Evidence Pack.

Exceptions

- Insufficient “before” data, show limited comparison.

Related UI

See UI-6: Strategy Management View

See UI-5: Strategy Effectiveness Dashboard

Use Case 6

Number: UC-6

Title: Export Evidence Pack Report

Preparer: Abdulrahman Al Hattali

Actor/User: Operations Executive (or QA/Operations Lead)

User Story

As an executive, I want to export an evidence-backed report so that I can share leadership-ready findings and justify recommendations with traceable call evidence.

Course of Events

1. User navigates to Reports & Export (UI-7).
2. The user selects a time window for the report.
3. The user applies filters (e.g., issue type, team/queue).
4. The user selects the report type Evidence Pack and clicks Generate Evidence Pack.
5. System compiles an **Evidence Pack artifact** that includes:
 - Time window definition
 - **Filters applied** (queue/issue/team)
 - Counts & trends with key operational metrics (AHT, FCR, escalation rate, sentiment)
 - Sample size
 - Representative call IDs (traceable examples supporting the claims)
 - Derived signals where relevant (e.g., sentiment shift, recurrence)
6. The user chooses an export format (PDF or CSV) and downloads the report.

Exceptions/Alternates:

- If there is insufficient data for the selected filters/time window, the system warns the user and suggests expanding the time window or broadening filters.

Related UI

See UI-7: Reports & Export View

Use Case 7

Number: UC-7

Title: Monitor Compliance in Customer Calls

Preparer: Anshul Kasera

Actor/User: Compliance Officer

User Story

As a compliance officer, I want to automatically detect whether agents follow required regulatory scripts so that the organization can ensure compliance with legal and company policies.

Course of Events

1. User navigates to Compliance Monitoring Dashboard.
2. System analyzes call transcripts for required compliance phrases and disclosures.
3. System displays:
 - Percentage of calls meeting compliance requirements
 - Calls missing mandatory disclosures
 - Compliance trend over time
4. The user selects a non-compliant call category.
5. The system displays example call transcripts with highlighted missing phrases.
6. User exports a compliance audit report.

Related UI

See UI-8: Compliance Dashboard

Use Case 8

Number: UC-8

Title: Reconstruct Customer Issue Journey

Preparer: Anshul Kasera

Actor/User: Customer Experience Manager

User Story

As a customer experience manager, I want to trace multiple calls from the same customer related to the same issue so that I can understand the full customer journey and identify service breakdowns.

Course of Events

1. The user navigates to Customer Journey Explorer.
2. User searches by customer ID or issue tag.
3. System groups related calls into a timeline view.
4. System displays:
 - Sequence of calls
 - Issue classification changes

- Sentiment progression
 - Escalation events
5. The user identifies where the issue was first reported and when it was resolved.
 6. The user exports the journey summary for customer experience analysis.

Related UI

See UI-9: Customer Journey Explorer

APPENDIX C: REVISIONS

Section	Changes
Executive Summary	Added call-level analysis and AR visualization description
Background	Minor wording improvements for clarity
Requirements Analysis	Expanded architecture with question extraction and call similarity engine
System Features	Added call-level retrieval and AR visualization features
Software Engineering Tools and Techniques	No changes
Timeline	New section added
UI / Use Case Appendix	No major changes
Other Appendices	Added Privacy and Ethics appendix
Personnel	No changes

APPENDIX D: Privacy, Data use and Ethics

The CI system is designed with a strong emphasis on data privacy, ethical AI usage, and enterprise compliance.

- All customer data is anonymized and masked during ingestion.
- The system operates in a closed, on-premise environment with no reliance on external APIs.
- Personally identifiable information (PII) is removed or obfuscated before processing.
- AI-driven insights are designed to be explainable and supported by traceable evidence.
- The system avoids biased decision-making by relying on aggregate statistical patterns rather than individual-level judgments.
- Access control mechanisms ensure that only authorized users can view sensitive data.

This approach ensures that CI adheres to both organizational policies and broader ethical standards in AI-driven analytics.